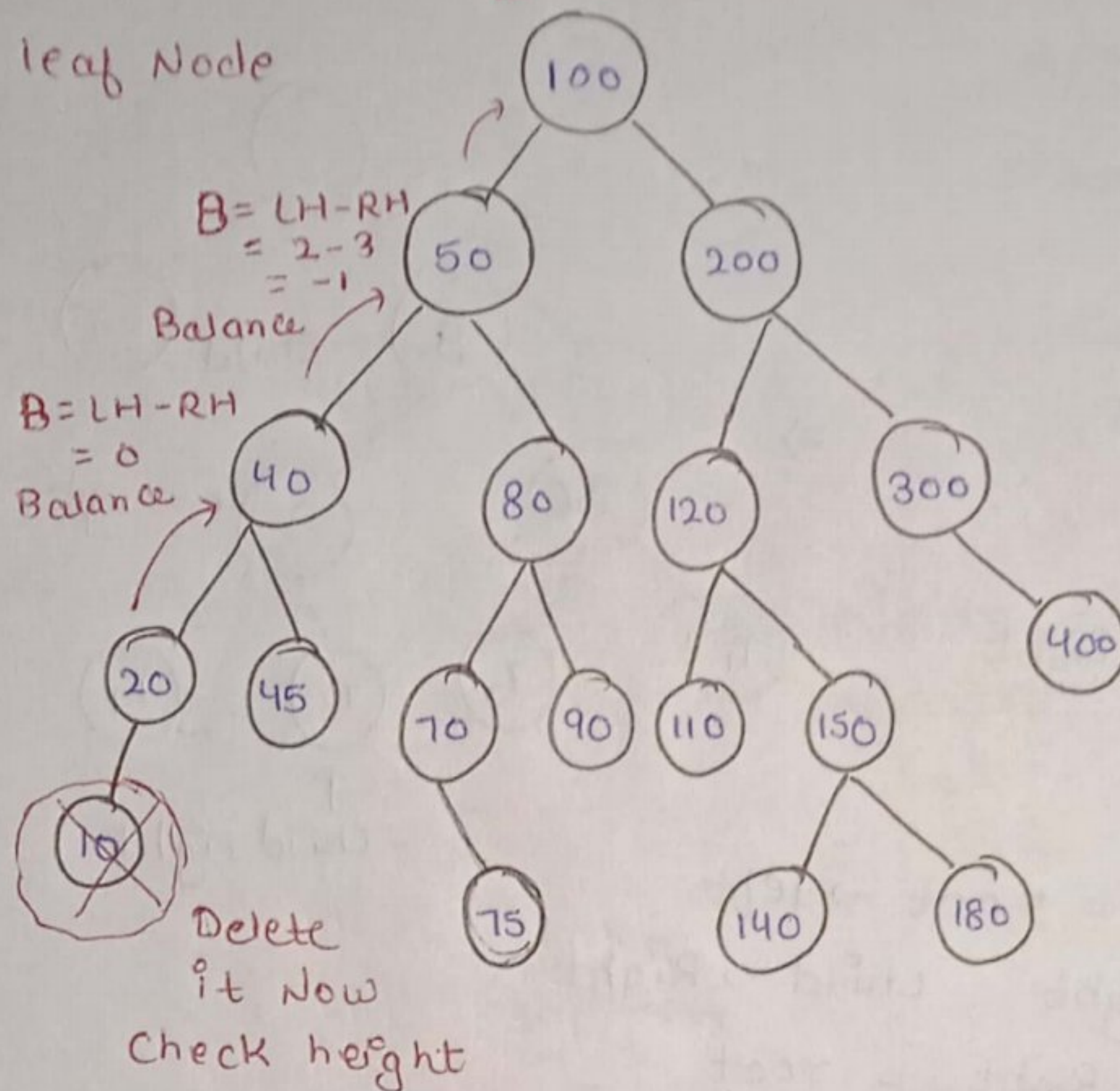


AVL TREE DELETION

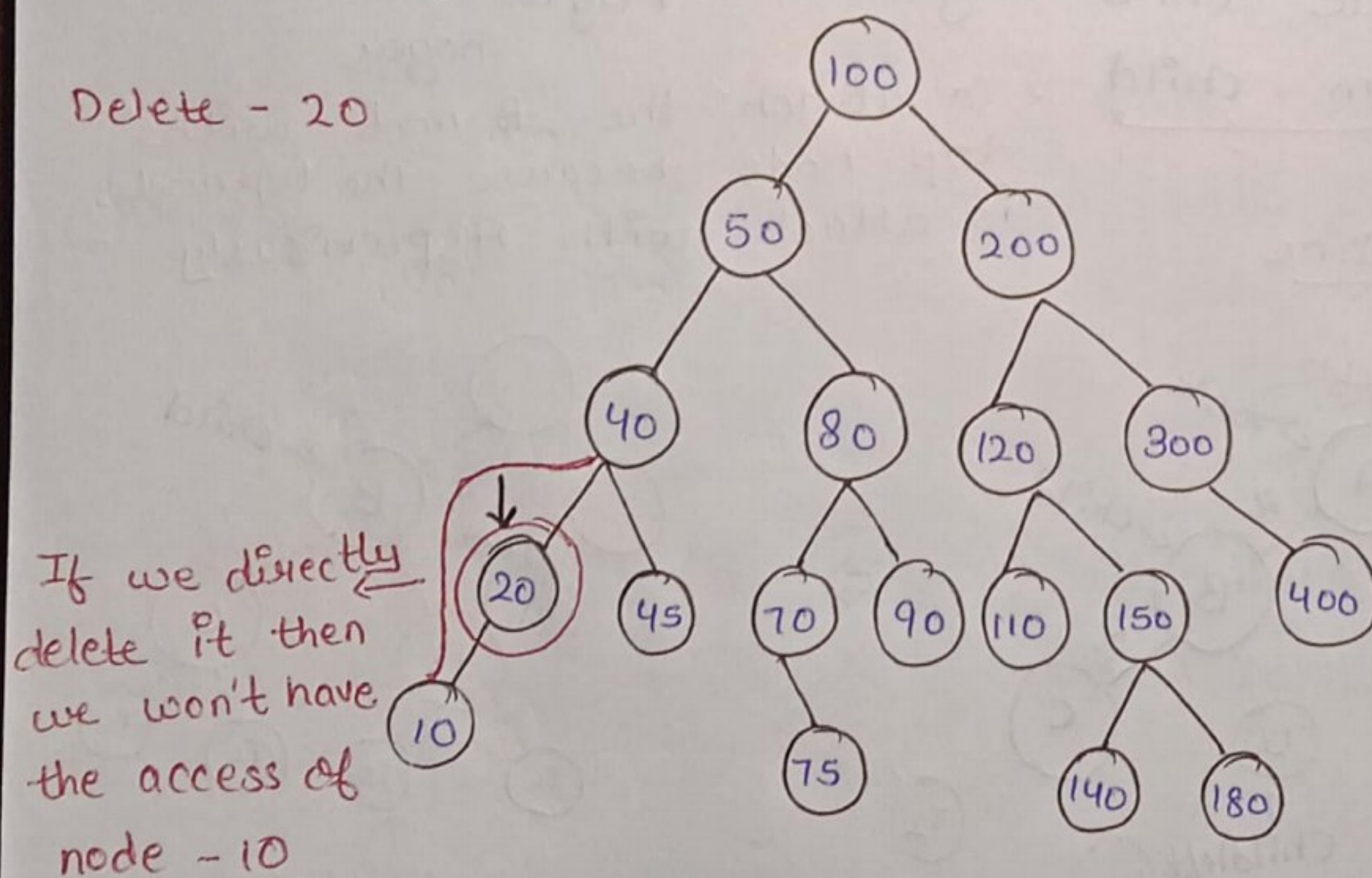
$$B = \overset{4}{8} - 4 = 0 \text{ (Balance)}$$

Delete leaf Node
→ 10



Delete a node whose only one child exist
either left or right

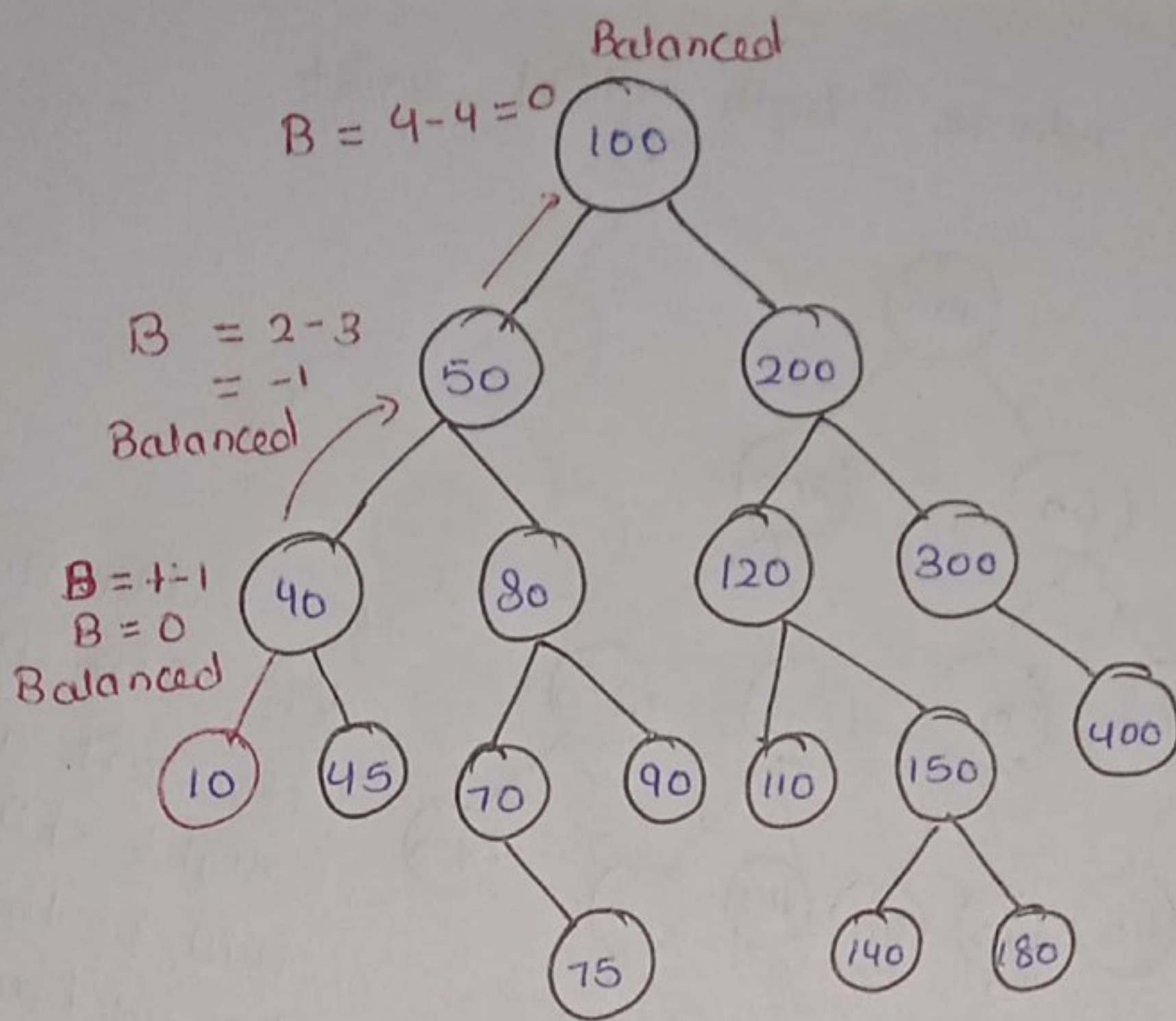
Delete - 20



If we directly delete it then we won't have the access of node - 10

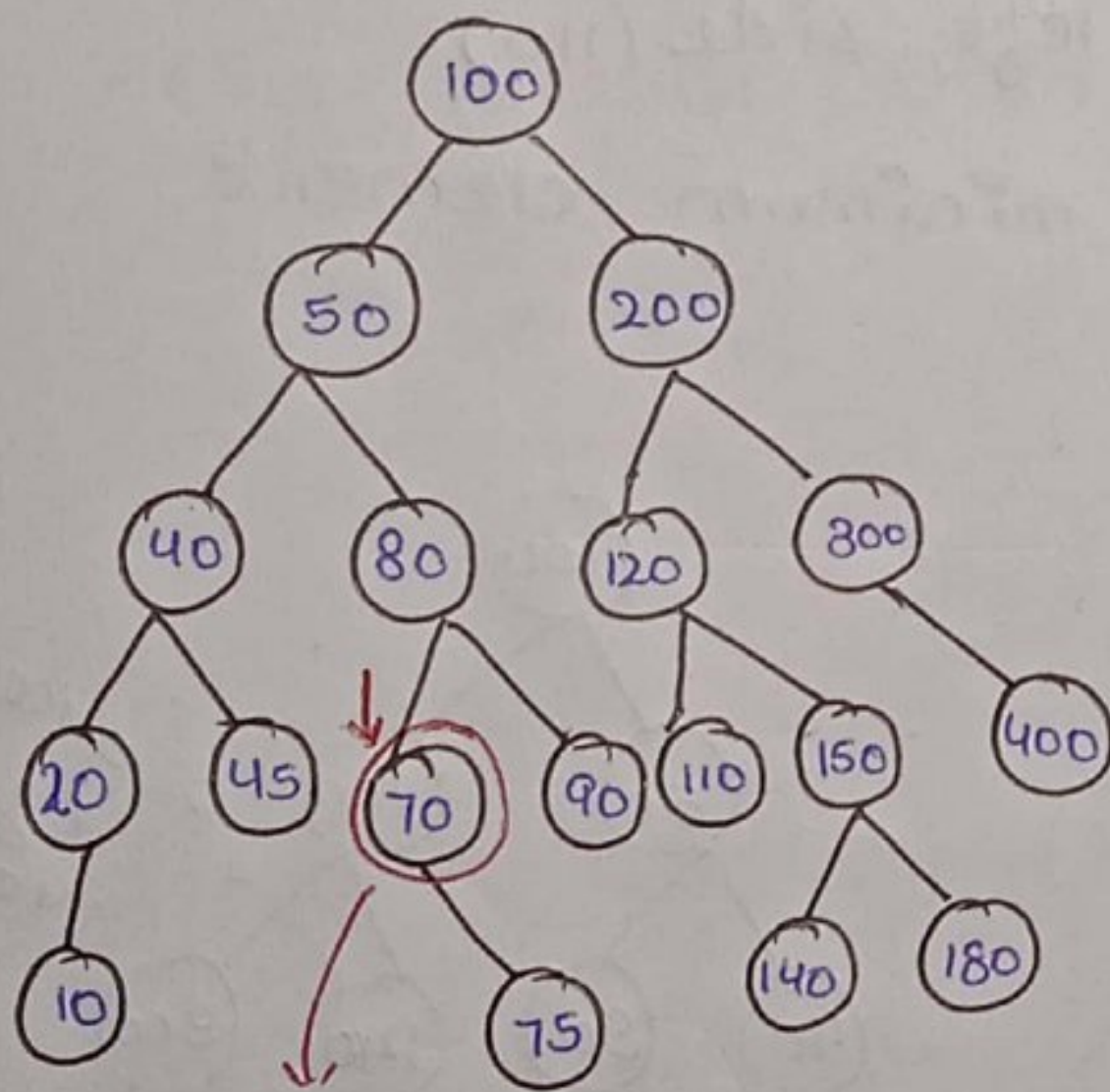
→ Delete that node

→ Return its left child

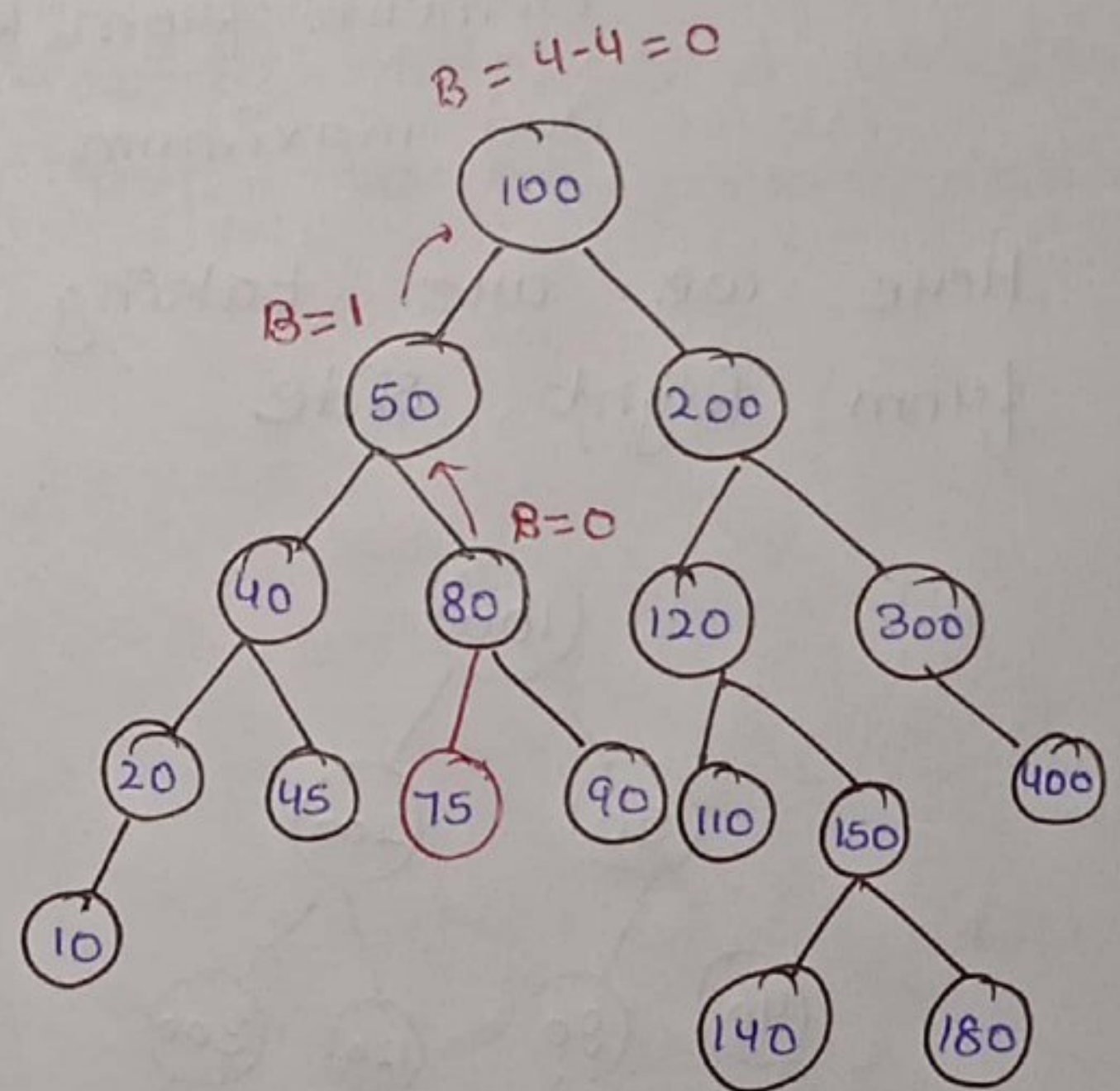


Deleting a node whose only right child exist

Delete - 70

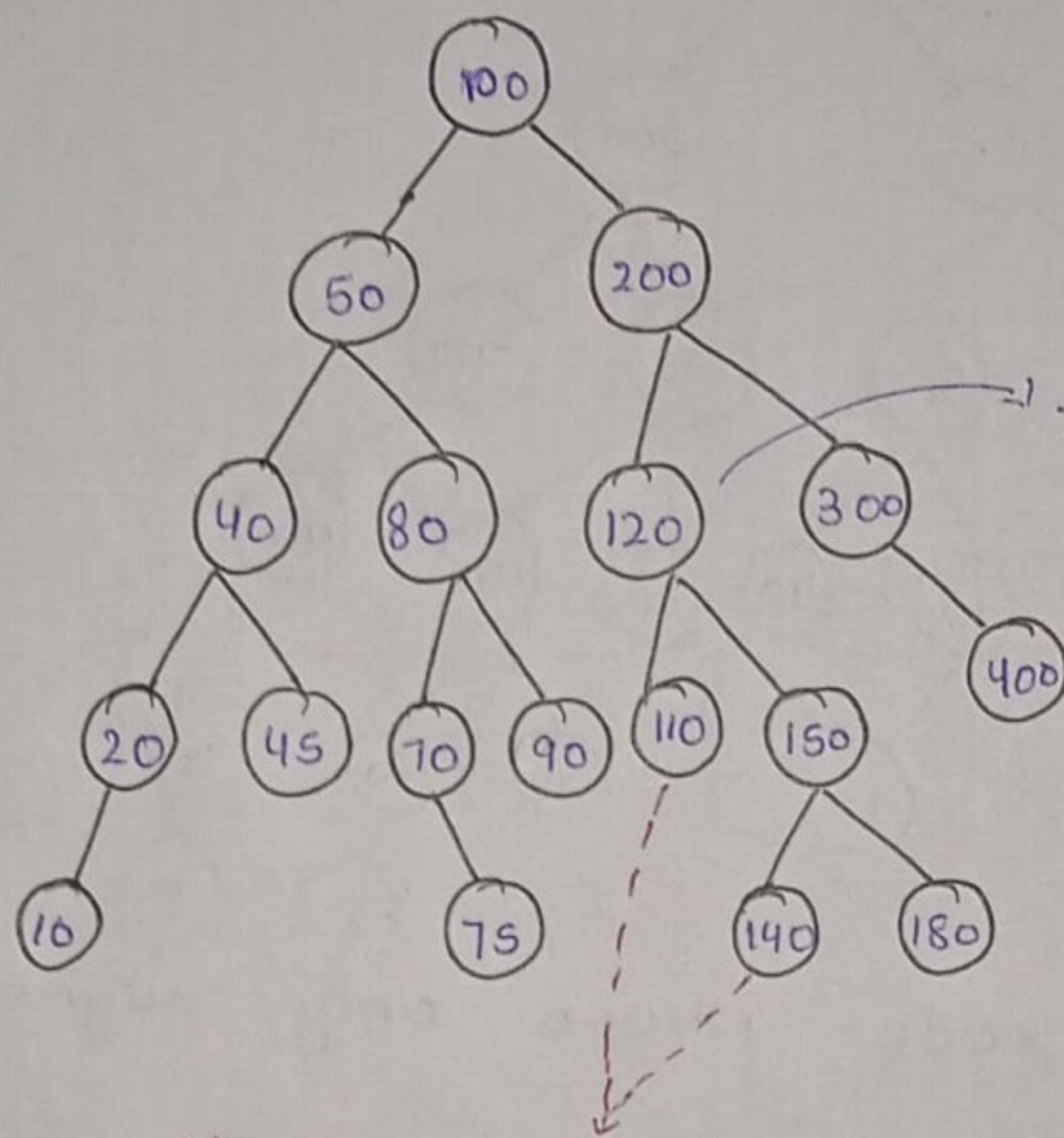


Delete this node
& return its right child



Deleting a node whose both child exist

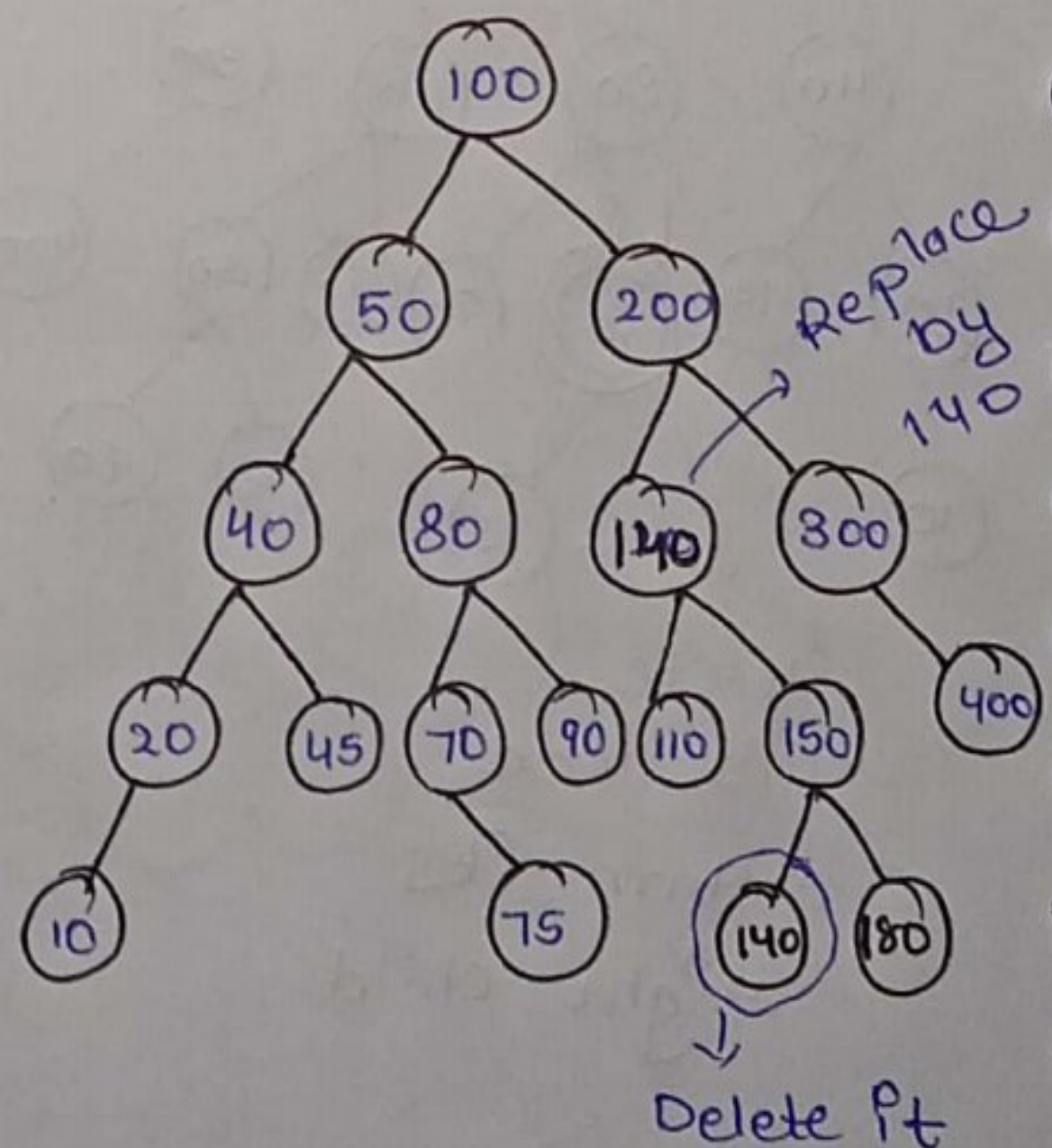
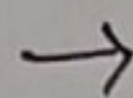
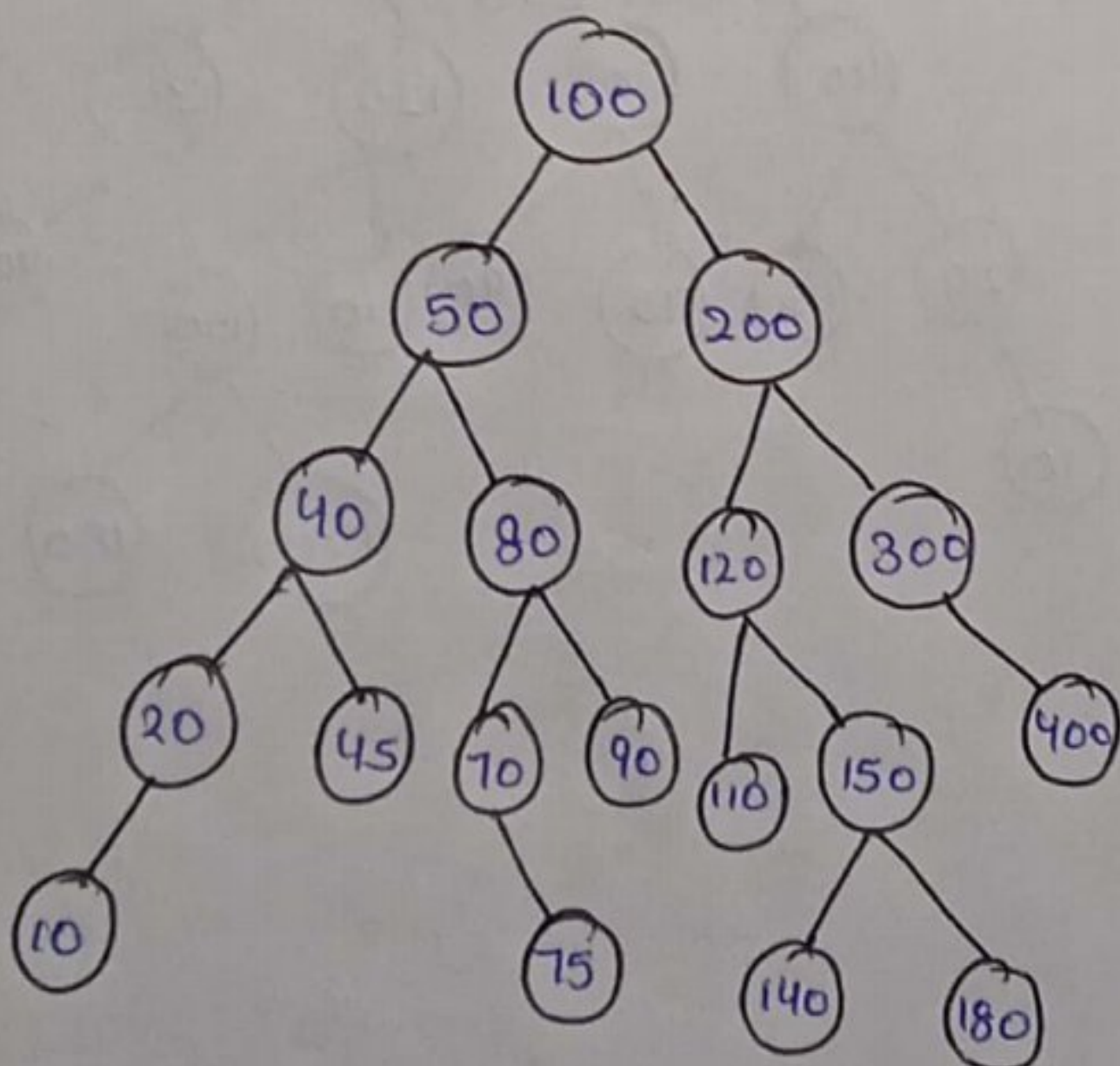
Delete - 120



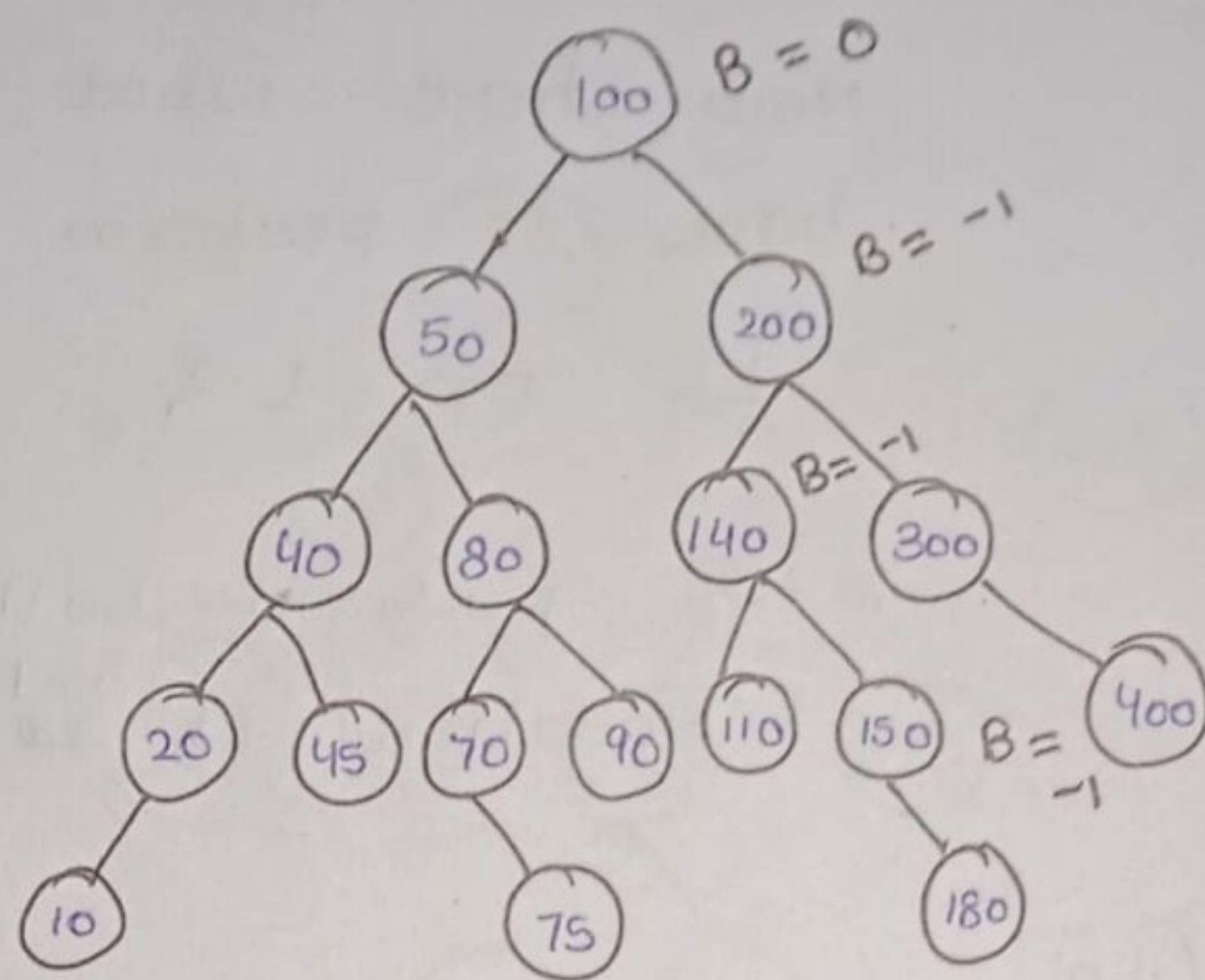
If we directly delete 120 then its left and right child will be lost so we have to replace it by any of its child node

We can take minimum element from ~~left~~ right side (140) and maximum from left side (110)

Here we are taking 140 minimum element from Right side



Replace the 120 by 140 and then delete 140

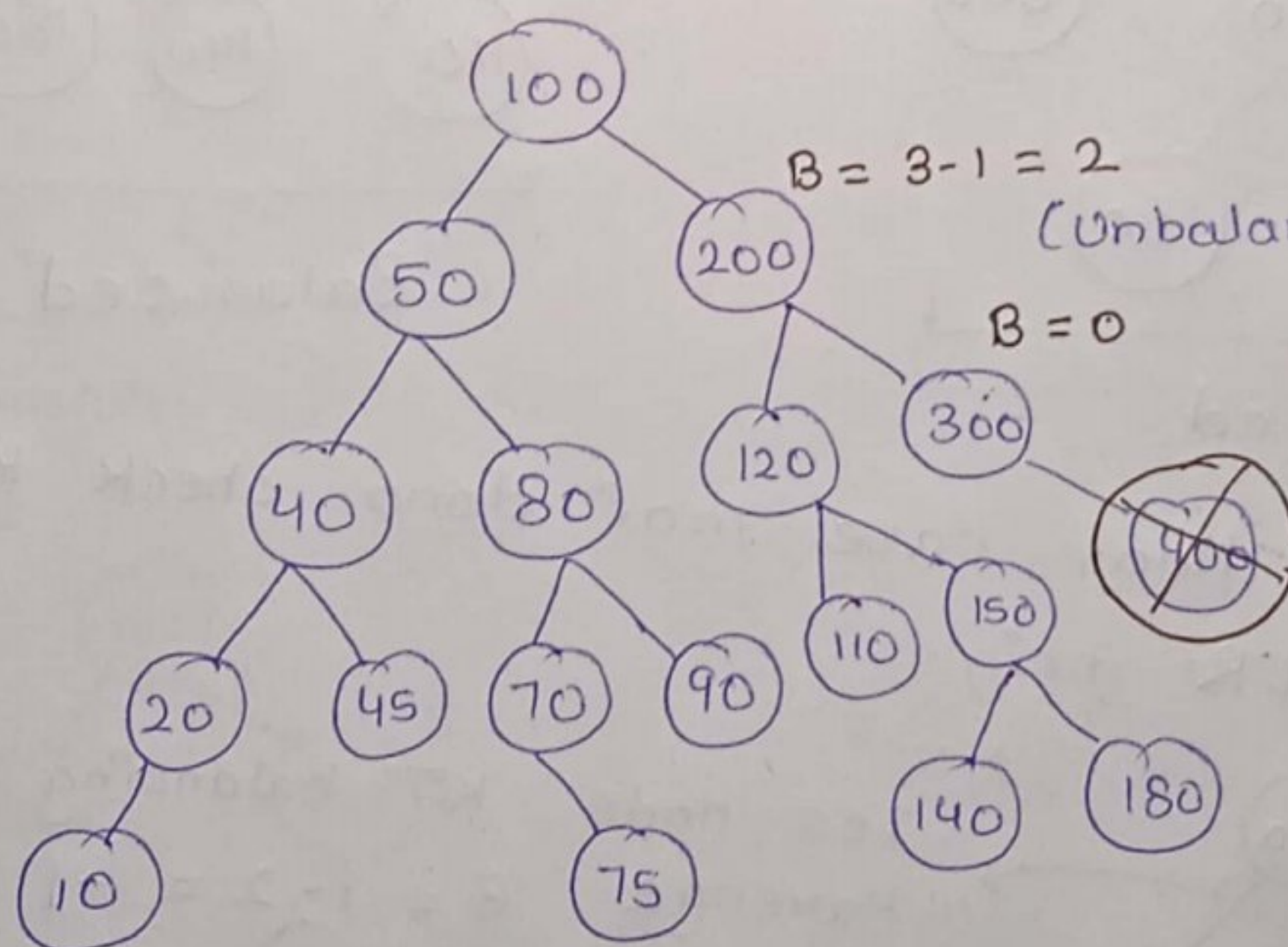


After deleting
node the tree
is balanced

Both child exist

- Right side find minimum (140)
- Replace it with root (120), minimum (140)
- Delete that node - 140

If we delete any particular node and unbalancing occur then we have to perform rotation



Delete - 400

(Unbalanced)

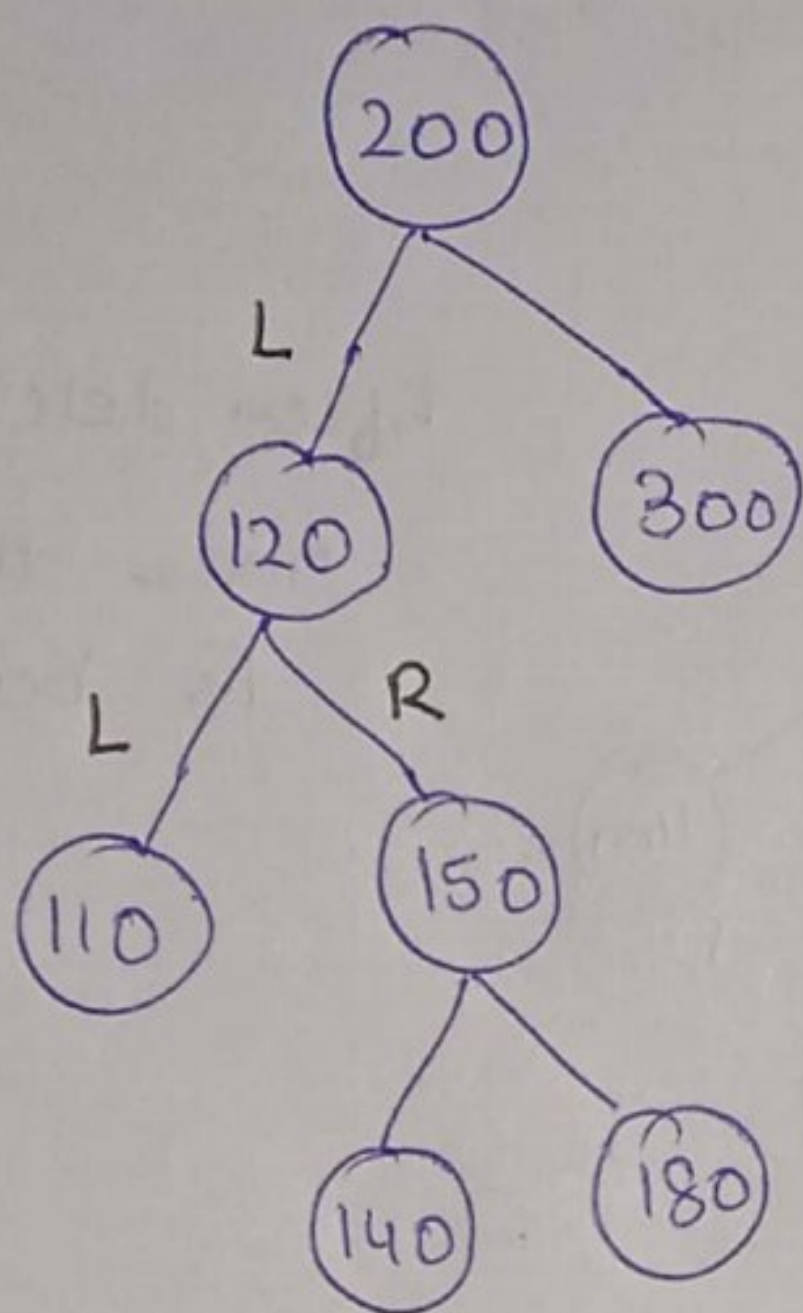
B=0

Now how
do we know
from which
side this
unbalancing
occur

→ Balance > 1
Unbalanced from left
side so we
perform LL or LR

→ Balance < -1
Unbalanced from right side
so RR or RL

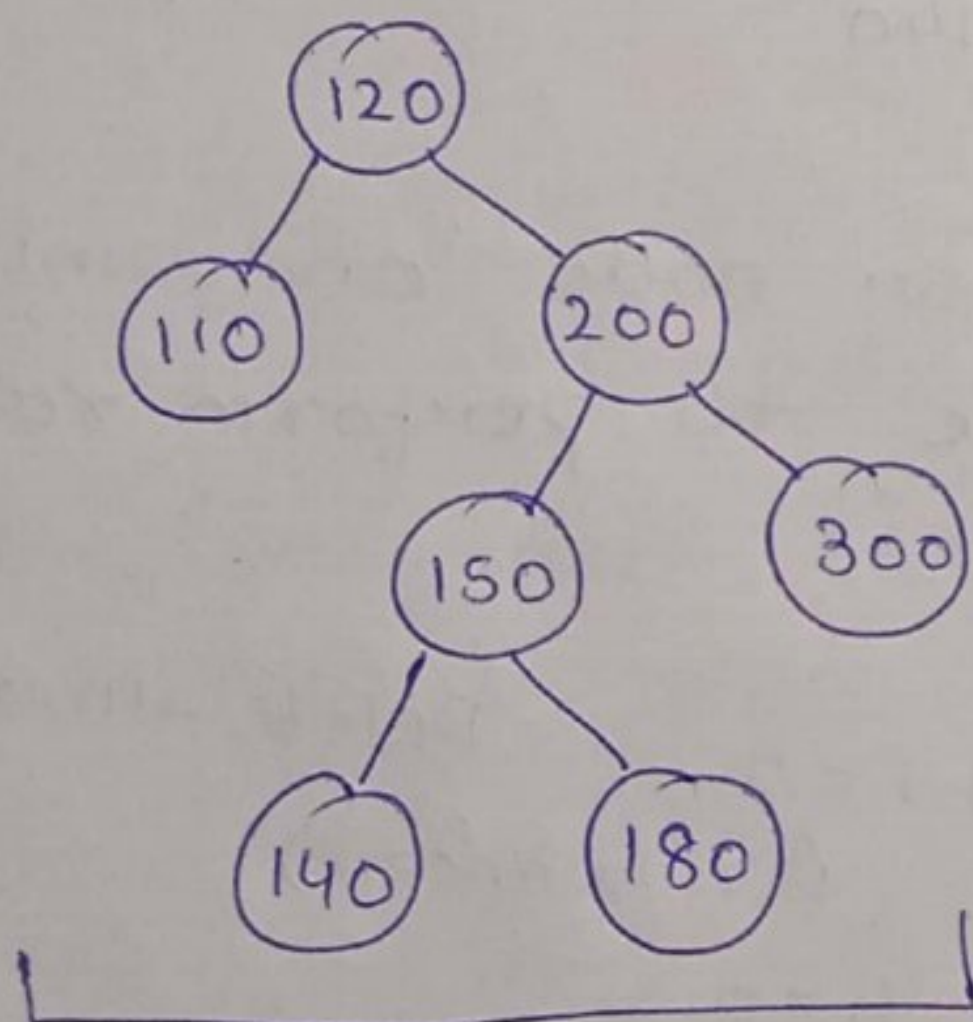




Now here what we have to perform either LL or LR

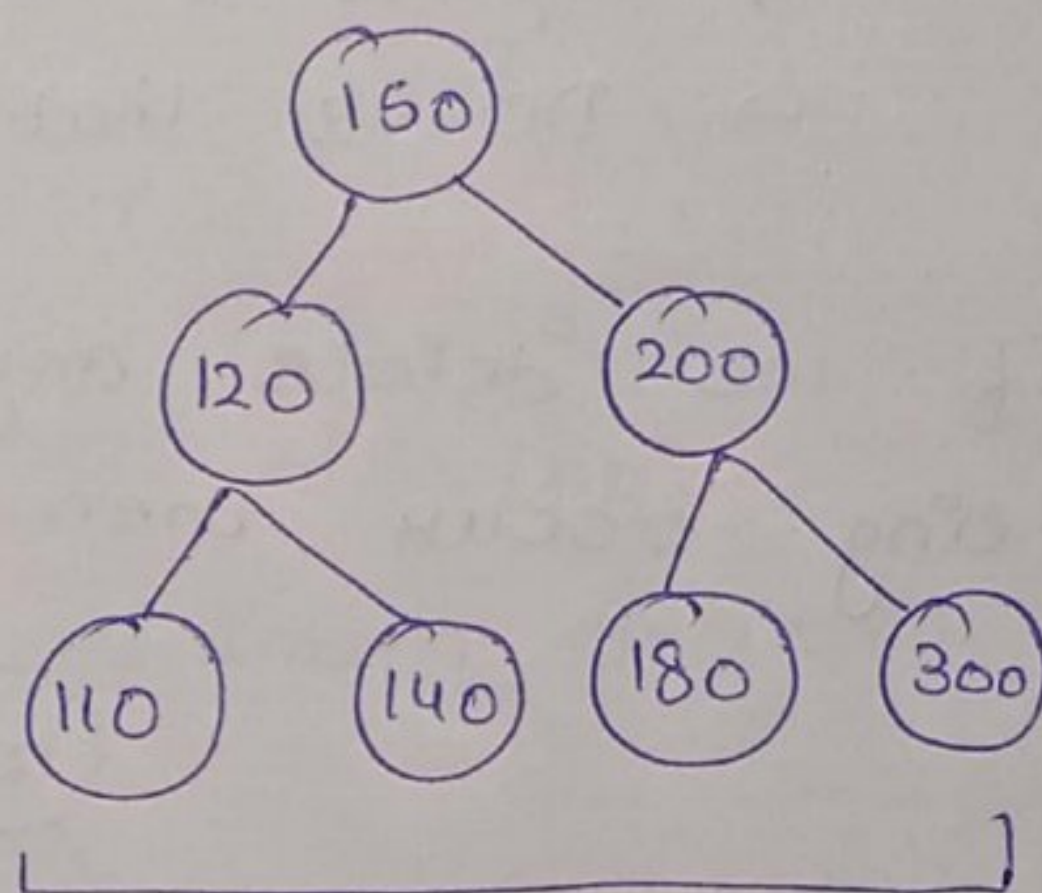
We perform both the rotation LL ~~and~~ LR

LL



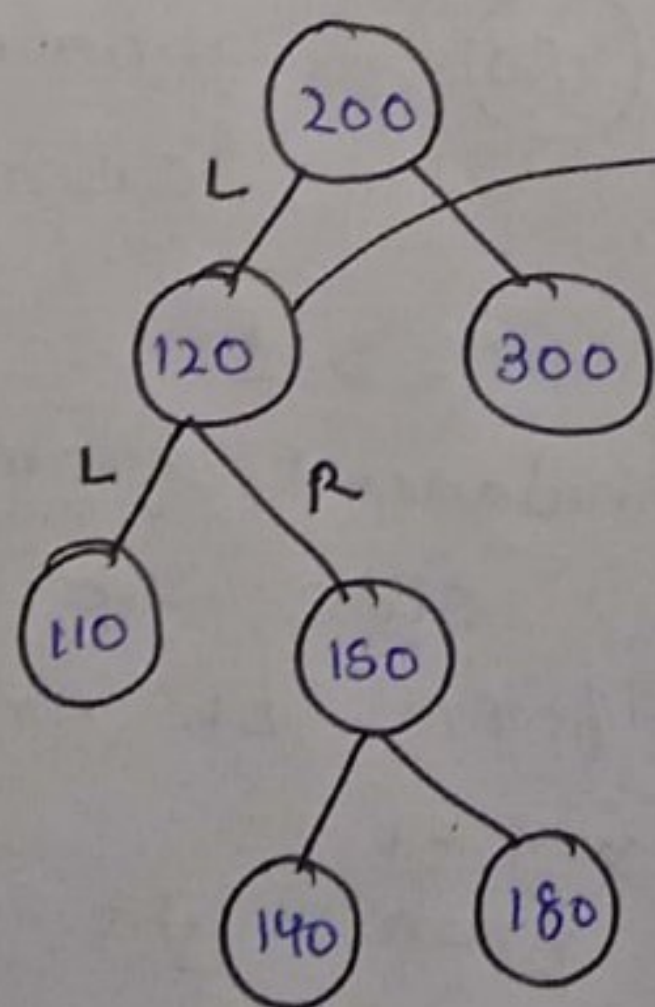
Unbalanced

LR



Balanced

Kya hum haar case mai dono check karenge LL or LR nhi



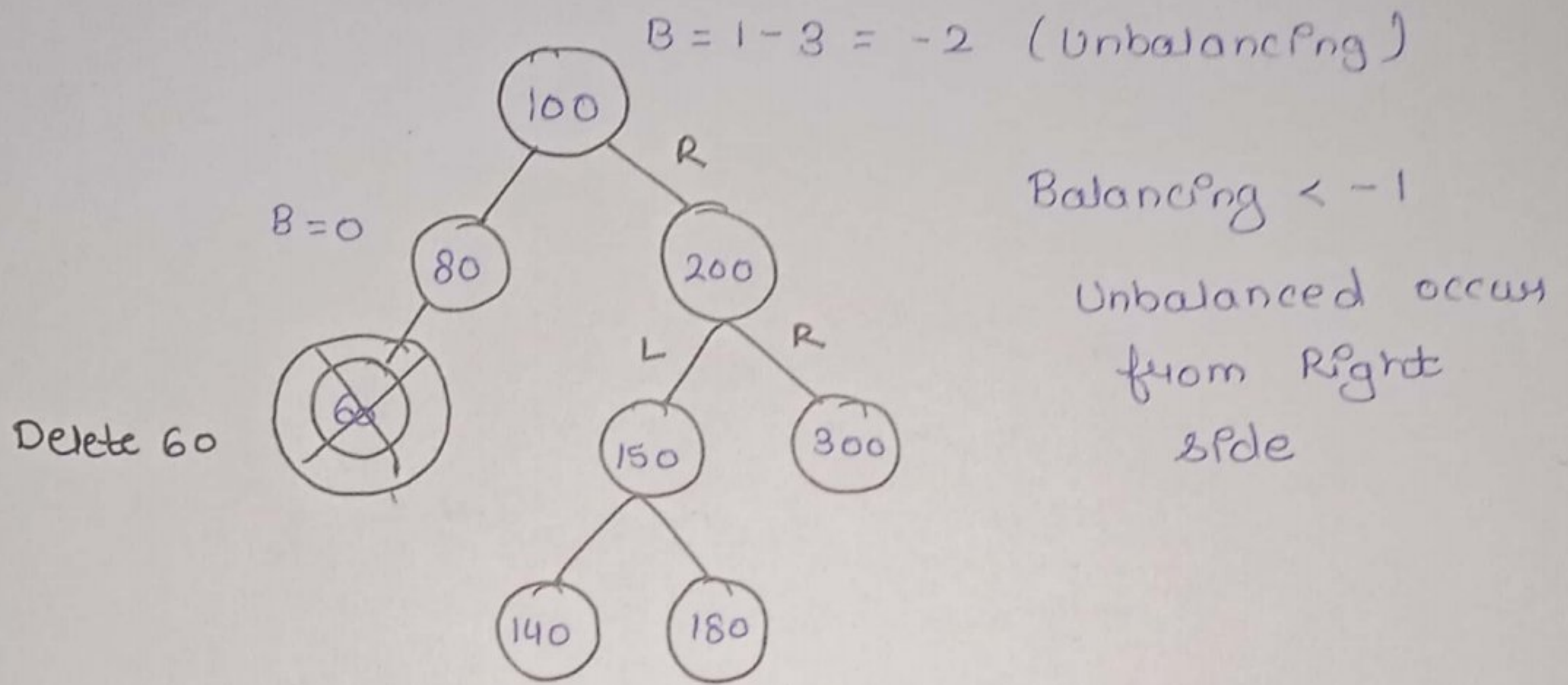
Iss node ki balancing check karenge $B = 1 - 2 = -1$

If Balancing < 0 then do LR

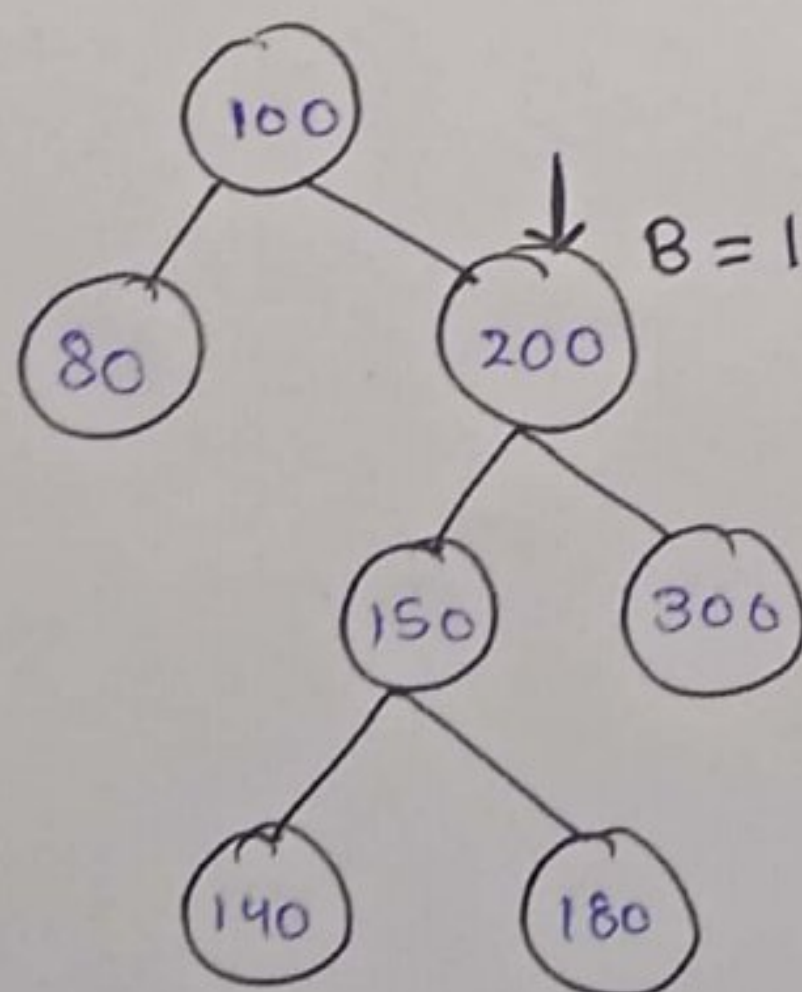
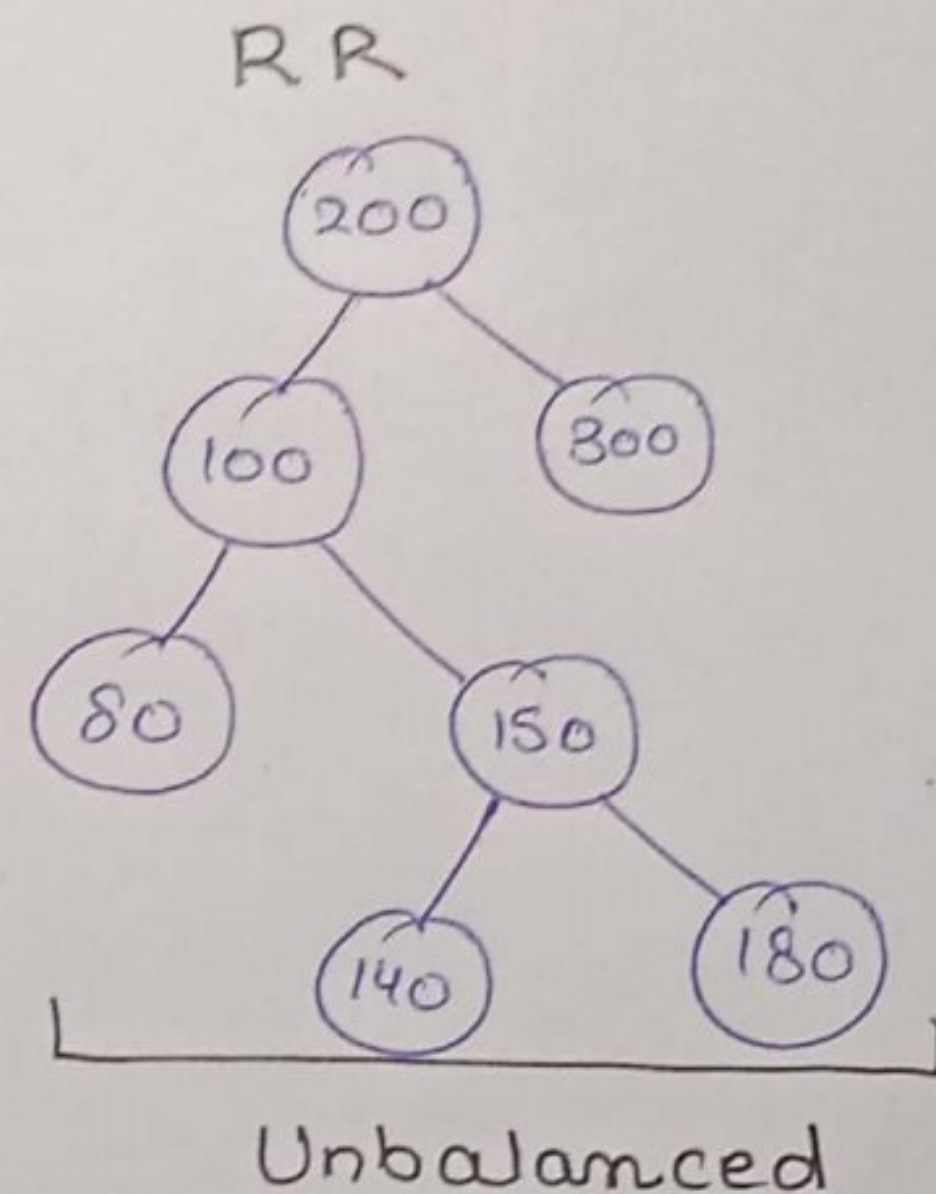
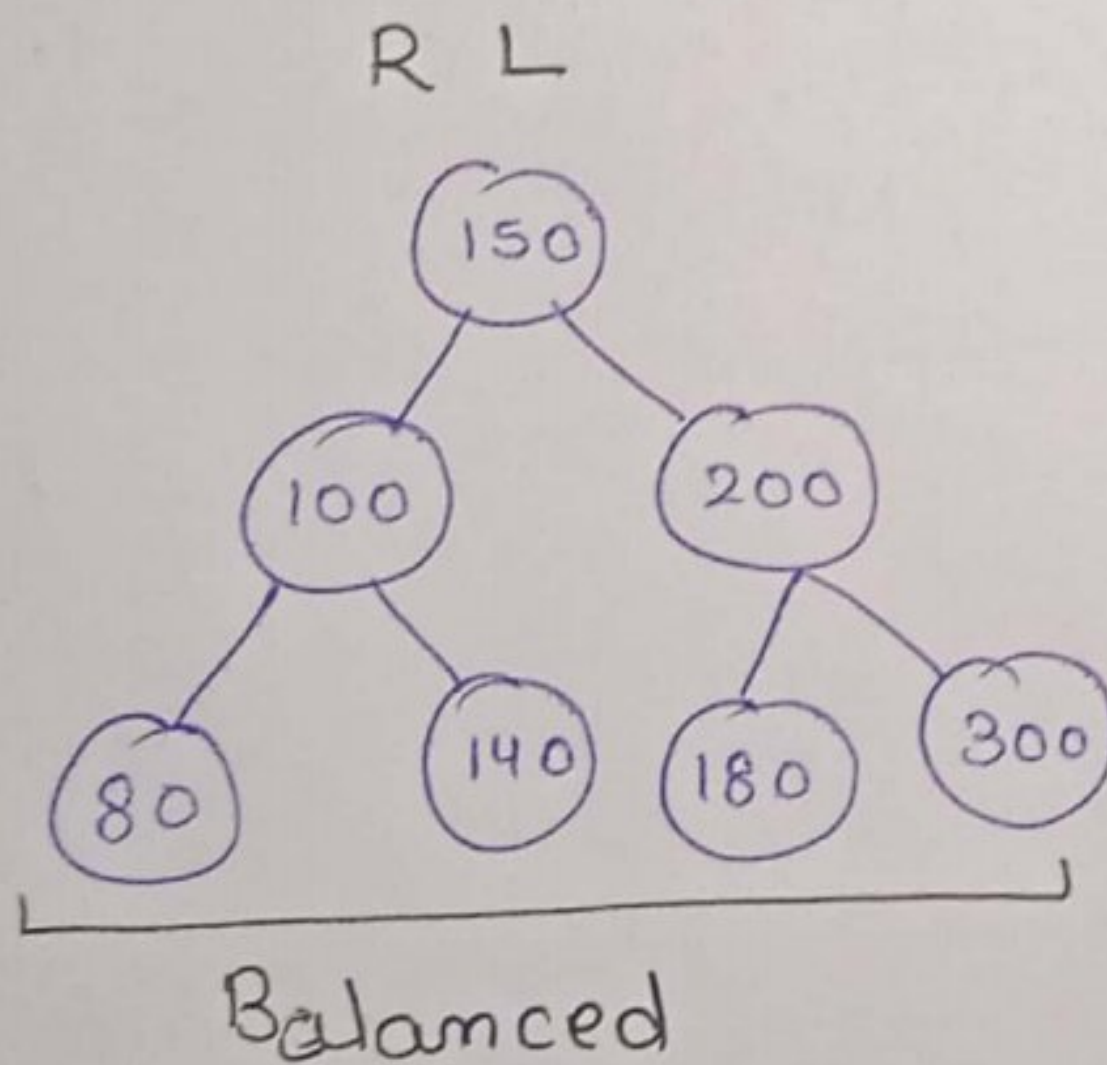
else

LL

Another Case



How do we know whether we have to perform RL or RR



If $B > 0$ then
do RL

else

RR

If $B == 0$ then we can
do any RR or RL but
we prefer RR because it's
a one step process

```
1  int getHeight(Node *root)
2  {
3      if (!root)
4      {
5          return 0;
6      }
7      return root->height;
8  }
9
10 int getBalance(Node *root)
11 {
12     return getHeight(root->left) - getHeight(root->right);
13 }
14
15 // Right Rotation
16 Node* rightRotation(Node *root)
17 {
18     Node *child = root->left;
19     Node *childRight = child->right;
20
21     child->right = root;
22     root->left = childRight;
23
24     // Update the Height
25     root->height = 1 + max(getHeight(root->left), getHeight(root->right));
26     child->height = 1 + max(getHeight(child->left), getHeight(child->right));
27
28     return child;
29 }
30
31 // Left Rotation
32 Node* leftRotation(Node *root) {
33     Node *child = root->right;
34     Node *childLeft = child->left;
35
```

```

36     child->left = root;
37     root->right = childLeft;
38
39     // Update the Height
40     root->height = 1 + max(getHeight(root->left), getHeight(root->right));
41     child->height = 1 + max(getHeight(child->left), getHeight(child->right));
42
43     return child;
44 }
45
46 Node* deleteNode(Node* root, int key)
47 {
48     //add code here,
49     if(!root)
50         return NULL;
51
52     if(key < root->data) //Left Side
53         root->left = deleteNode(root->left, key);
54
55     else if(key > root->data) //Right Side
56         root->right = deleteNode(root->right, key);
57
58     else
59     {
60         //Leaf Node
61         if(!root->left && !root->right)
62         {
63             delete root;
64             return NULL;
65         }
66
67         //Only one child exist
68         else if(root->left && !root->right) //Left child
69         {
70             Node *temp = root->left;
71             delete root;
72             return temp;
73         }

```

```

75     else if(!root->left && root->right)
76     {
77         Node *temp = root->right;
78         delete root;
79         return temp;
80     }
81
82     //Both Child exist
83     else
84     {
85         // Right side smallest element
86         Node *curr = root->right;
87
88         while(curr->left)
89         {
90             curr = curr->left;
91         }
92
93         root->data = curr->data;
94         root->right = deleteNode(root->right, curr->data);
95     }
96 }
97
98 //Update the height
99
100 root->height = 1+max(getHeight(root->left), getHeight(root->right));
101
102 // Check the balance
103 int balance = getBalance(root);
104

```

```
105 //left side
106 if(balance > 1)
107 {
108
109     if(getBalance(root->left) >= 0)
110     return rightRotation(root);
111
112     else
113     {
114         root->left = leftRotation(root->left);
115         return rightRotation(root);
116     }
117 }
118
119 //right side
120 else if(balance < -1)
121 {
122
123     if(getBalance(root->right) <= 0)
124     return leftRotation(root);
125
126     else
127     {
128         root->right = rightRotation(root->right);
129         return leftRotation(root);
130     }
131
132
133 }
134
135 else
136 return root;
137
138 }
```